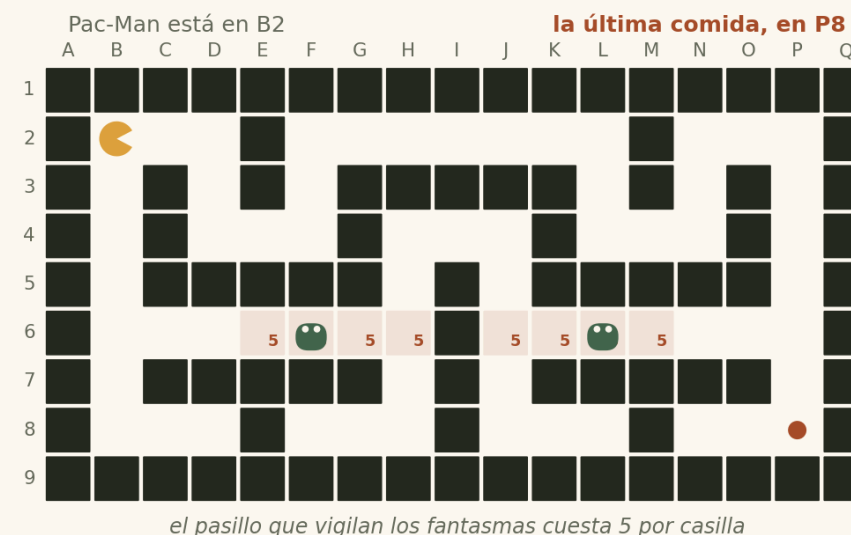


Buscar es probar en orden

Cómo resuelve una máquina un problema del que nadie le dio la receta: probando caminos, pero en un orden que se puede justificar.

Pac-Man no sabe dónde está la comida. Solo sabe a qué casillas puede moverse desde donde está. Con eso basta.

Treinta y ocho de los sesenta minutos son de ustedes: tres estaciones con la simulación abierta y el Laboratorio 2.



El laberinto de la clase: Pac-Man en B2, la comida en P8

Cómo vamos a trabajar hoy

Dos explicaciones cortas y tres estaciones.

- 01 Formular un problema • 8 minutos, con un ejemplo resuelto
- 02 Estación 1 • formulan el suyo, 12 minutos
- 03 Estación 2 • recorren a mano y comprueban, 16 minutos
- 04 Estación 3 • costo uniforme, 10 minutos

Abran la simulación antes de empezar: después funciona sin internet, también en el celular.

Un problema, cinco piezas

Mientras falte una de las cinco, no hay problema que buscar: hay una idea vaga.

- 01 Estado inicial • dónde empieza
- 02 Acciones • qué puede hacer desde cada estado
- 03 Modelo de transición • a qué estado lleva cada acción
- 04 Prueba de meta • cómo sabe que llegó
- 05 Costo de camino • cuánto cuesta cada paso

El espacio de estados no se escribe: se deduce de las cuatro primeras piezas. Este laberinto tiene 64 casillas libres; el Pac-Man de verdad pasa de los millones de estados.

El laberinto, formulado

El mismo tablero de la primera lámina, escrito como problema de búsqueda.

Pieza	En este problema
Estado inicial	Pac-Man en la casilla B2
Acciones	Moverse arriba, abajo, izquierda o derecha
Transición	Queda en la casilla contigua, salvo que haya muro
Prueba de meta	¿Está en P8, donde quedó la última comida?
Costo de camino	1 por casilla; 5 si la vigila un fantasma

Ese 5 no viene del juego original: lo ponemos nosotros para que el costo importe. Sin él, las tres estrategias que veremos se reducen a dos.

Formulen el suyo

Doce minutos, en parejas. Un problema de su entorno, no del juego.

- 01 Elijan algo con estados discretos: rutas de combi, turnos de un consultorio, el orden de un trámite
- 02 Escriban las cinco piezas, cada una en una línea
- 03 Estimen cuántos estados tendría: si no pueden, el problema todavía es vago
- 04 Marquen qué acción es la más cara y por qué

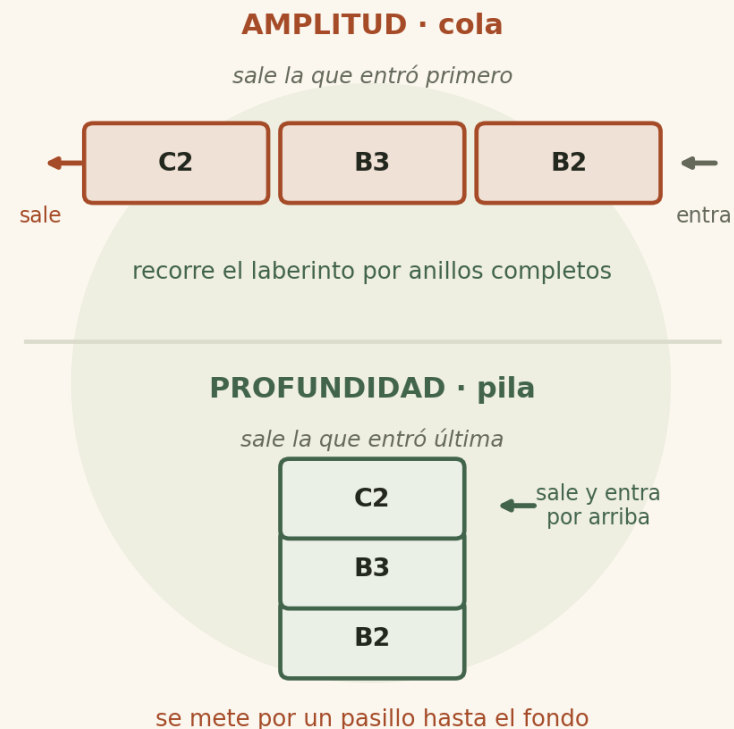
Si al escribir la prueba de meta tienen que decir «cuando esté más o menos bien», falta acotar el problema: eso es una pregunta abierta, no una meta.

Amplitud: la frontera es una cola

Expande primero la casilla más antigua. Se abre en anillos alrededor de Pac-Man.

Desde B2 entran B3 y C2, en ese orden. Solo cuando termina con esas dos pasa a las que ellas abrieron.

Encuentra siempre el camino con menos pasos. Lo paga en memoria: guarda todo un anillo a la vez.



La frontera: cola en amplitud, pila en profundidad

Profundidad: la frontera es una pila

Expande primero la casilla más reciente. Se mete por un pasillo hasta el fondo antes de probar otro.

Desde B2 entran B3 y C2, pero sale la última: C2. De ahí sigue por D2, D3, D4... hasta chocar con un muro; B3 espera al fondo de la pila.

Gasta poquísima memoria: solo el pasillo actual. A cambio, el camino que encuentra puede ser cualquiera.

Sin memoria de lo ya visto, $B2 \rightarrow C2 \rightarrow B2 \rightarrow C2$ es un ciclo perfectamente válido para el algoritmo.

Las dos, comparadas

Cuatro preguntas que se le hacen a cualquier estrategia de búsqueda.

Pregunta	Amplitud	Profundidad
¿Encuentra solución si existe?	Sí	No, si el espacio no tiene fondo
¿La de menos pasos?	Sí	No
Memoria	Todo un anillo	Solo el pasillo actual
Cuándo conviene	La meta está cerca	Las soluciones son profundas

Ninguna de las dos mira lo que cuesta cada casilla: cuentan pasos, no riesgo.

Predigan, después comprueben

Dieciséis minutos. Primero el papel, después la pantalla.

- 01 Escriban las seis primeras casillas que expandirá amplitud desde B2
- 02 Hagan lo mismo con profundidad; el orden de vecinos es siempre arriba, abajo, izquierda, derecha
- 03 Recién entonces abran la simulación y comparen paso a paso
- 04 Anoten dónde se equivocaron: eso es lo que va en el informe

El orden en que entran los vecinos cambia el resultado de profundidad y no cambia el de amplitud. Compruébenlo, no lo crean.

Casillas repetidas: árbol contra grafo

La misma búsqueda, con y sin memoria de lo ya visitado.

	Búsqueda en árbol	Búsqueda en grafo
Qué recuerda	Nada	Las casillas ya expandidas
B2 → C2 → B2	Lo vuelve a expandir	Lo descarta
Riesgo	Da vueltas para siempre	Más memoria ocupada

En el laboratorio, la lista de visitados es la diferencia entre un programa que termina y uno que hay que matar desde la terminal.

Costo uniforme: cuando los pasos cuestan distinto

Expande primero la casilla de menor costo acumulado, no la más antigua ni la más reciente.

Estrategia	Saca de la frontera	Optimiza
Amplitud	La más antigua	El número de pasos
Profundidad	La más reciente	Nada
Costo uniforme	La de menor costo acumulado	El costo total del camino

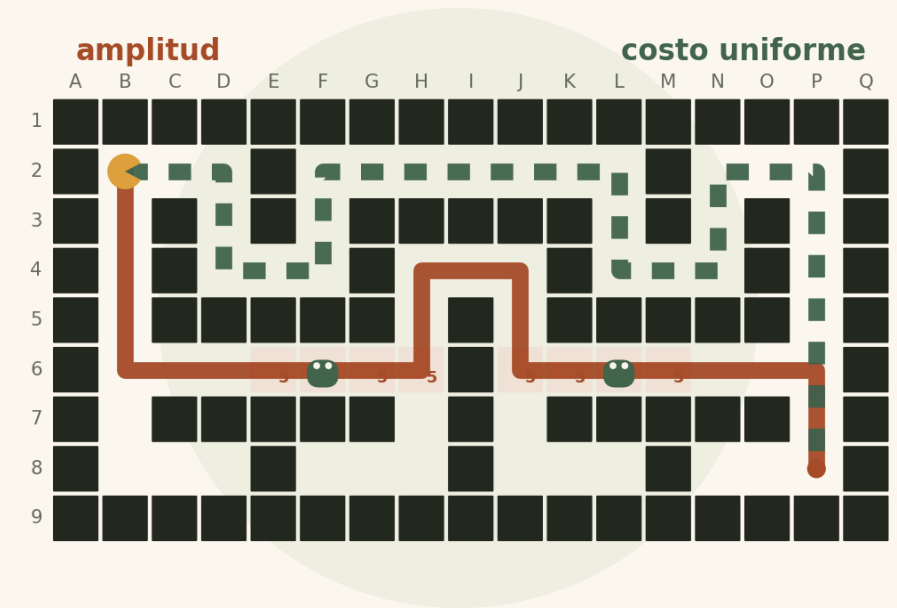
Si los fantasmas se fueran y todas las casillas costaran 1, costo uniforme y amplitud darían exactamente el mismo camino. La diferencia la hacen los fantasmas.

Menos pasos no es más barato

Diez minutos. El mismo laberinto, ahora contando lo que cuesta cada casilla.

Amplitud cruza el pasillo vigilado: 24 pasos, pero cuesta 56. Costo uniforme lo rodea por arriba: 28 pasos y cuesta 28.

Pongan la simulación en costo uniforme, lleguen al final y expliquen en una frase por qué amplitud se equivoca aquí, si es la que encuentra el camino más corto.



El pasillo vigilado: uno lo cruza, el otro lo rodea

Laboratorio 2 • lo que se entrega

BFS y DFS programados, con la comparación medida.

Parte	Qué tiene que contener
Código	bfs() y dfs() en Python, con lista de visitados
Problemas	El laberinto de la clase y el suyo, el de la estación 1
Medición	Casillas expandidas y longitud del camino, por estrategia
Discusión	Por qué difieren, y dónde falló su predicción de la estación 2

Se entrega en el aula virtual, en un cuaderno de Jupyter o un repositorio. Declaren al final qué asistentes usaron y para qué.

De dónde sale todo esto

Lo de hoy es capítulo 3 de un libro y el proyecto 1 de un curso.

Fuente	Qué buscar ahí
Russell y Norvig, AIMA (4.^a ed.)	Capítulo 3: formulación, amplitud, profundidad y costo uniforme
Curso CS188 • Berkeley	Proyecto 1: Pac-Man resuelto con estas tres estrategias, con código de partida
Documentación de Python: <code>collections.deque</code>, <code>heapq</code>	Las estructuras exactas del laboratorio

La próxima sesión: árboles y grafos de problemas, con la complejidad en tiempo y espacio de cada estrategia. Traigan el laboratorio corriendo.